

Blatt 6

Grundlagen der IT-Sicherheit

Luis Staudt

Aufgabe 1: Einmalpasswörter (TAN-Liste)

a) Sicherheit bei Mithören eines genutzten Passworts

Jedes Einmalpasswort ist nach einmaliger Nutzung entwertet. Selbst wenn ein Angreifer ein bereits verwendetes Passwort mithört oder kopiert, kann er damit keine weitere Überweisung tätigen. Der Server akzeptiert jedes Passwort nur ein einziges Mal. Ein Replay-Angriff ist daher wirkungslos.

b) Sicherheit bei Abfangen von Datenpaketen

Nein, das Verfahren wäre dann **nicht mehr sicher**. Wenn der Angreifer Datenpakete abfangen (und nicht nur mitlesen) kann, könnte er folgendes tun:

- Das Datenpaket mit dem gültigen Einmalpasswort abfangen, sodass es den Server nie erreicht.
- Das Einmalpasswort ist dann noch nicht entwertet, da der Server es nie erhalten hat.
- Der Angreifer kann das abgefangene Passwort nun selbst verwenden, um eine eigene Überweisung durchzuführen (Man-in-the-Middle-Angriff).

c) Gegenmaßnahme

- **Transaktionsbindung:** Das Einmalpasswort wird an die konkreten Überweisungsdaten gebunden (Empfänger, Betrag, IBAN). Der Server prüft, ob das Passwort zu den eingereichten Transaktionsdaten passt. Selbst wenn der Angreifer das Passwort abfängt, kann er es nicht für eine andere Überweisung verwenden.
- **Verschlüsselter Kanal (TLS):** Durch Ende-zu-Ende-Verschlüsselung wird das Abfangen und Manipulieren von Paketen verhindert.
- **Zeitbegrenzung:** Das Einmalpasswort ist nur für wenige Minuten gültig, was das Zeitfenster für einen Angriff stark einschränkt.

d) Verifikation über Bankkarte (chipTAN)

Funktionsweise:

1. Der Nutzer gibt eine Überweisung im Online-Banking ein.
2. Die Überweisungsdaten (Empfänger-IBAN, Betrag) werden an die EC-Karte übertragen, entweder optisch über einen Flickercode am Bildschirm oder durch manuelle Eingabe am TAN-Generator.
3. Die EC-Karte berechnet mit ihrem internen geheimen Schlüssel und den Überweisungsdaten ein Einmalpasswort (TAN).
4. Der Nutzer gibt die angezeigte TAN im Browser ein.
5. Der Server berechnet mit dem ihm bekannten Schlüssel dieselbe TAN und vergleicht.

Warum ist das sicher?

- Die TAN ist an die konkreten Transaktionsdaten gebunden. Verändert ein Angreifer die Überweisung (z. B. anderen Empfänger), stimmt die TAN nicht mehr.
- Der geheime Schlüssel verlässt nie die EC-Karte. Selbst bei kompromittiertem Computer kann der Angreifer den Schlüssel nicht extrahieren.
- Der Nutzer sieht die Überweisungsdaten auf dem TAN-Generator und kann Manipulation erkennen, bevor er die TAN eingibt.

Aufgabe 2: Challenge-Response-Verfahren

a) Warum Challenge-Response?

Das Verfahren wird genutzt, weil das Passwort **nie über das Netzwerk übertragen** wird. Stattdessen wird nur der Hashwert aus Challenge und Passwort gesendet. Ein Angreifer, der den Netzwerkverkehr mithört, sieht nur die Challenge und die Response, kann daraus aber nicht das Passwort rekonstruieren (Einwegeigenschaft des Hashes). Außerdem ist jede Response einzigartig, da jede Challenge ein anderer Zufallswert ist. Ein Replay-Angriff mit einer abgefangenen Response ist daher nutzlos.

b) Mehrfache Nutzung derselben Challenge

Wenn dieselbe Challenge mehrfach verwendet wird, ist die Response bei gleichem Passwort jedes Mal identisch. Das verringert die Sicherheit erheblich:

- Ein Angreifer könnte eine korrekte Response aufzeichnen und bei der nächsten gleichen Challenge wiederverwenden (**Replay-Angriff**).
- Der Angreifer muss das Passwort nicht kennen, sondern nur die gültige Response zur bekannten Challenge.

c) Schutz vor Challenge-Wiederholung

- **Zufällige Challenges:** Jede Challenge wird kryptographisch zufällig generiert. Bei ausreichender Länge (z. B. 128 Bit) ist eine Wiederholung extrem unwahrscheinlich.
- **Zeitstempel einbeziehen:** Die Challenge enthält einen Zeitstempel. Der Server akzeptiert nur Responses zu aktuellen Zeitstempeln und lehnt veraltete ab.
- **Nonce-Speicher:** Der Server speichert bereits verwendete Challenges und lehnt jede erneut eingehende ab.

d) Beispielhafter Ablauf

Parameter:

- Hashfunktion: SHA-256
- Gemeinsames Geheimnis (Passwort): `passwort123`

Ablauf:

1. **Client** sendet Login-Anfrage an den Server.
2. **Server** generiert eine zufällige Challenge und sendet sie an den Client:

```
1 Challenge = X7kP9mQ2
2
```

3. **Client** berechnet die Response:

```
1 Response = SHA-256( "X7kP9mQ2" + "passwort123" )
2           = SHA-256( "X7kP9mQ2passwort123" )
3           = D1A9107BFFC537598B50769DC0883506
4           = D2F79F0E23BA4D3F22AF005C878DD1C4
5
```

4. **Client** sendet die Response an den Server.
5. **Server** berechnet mit dem ihm bekannten Passwort denselben Hash und vergleicht. Da beide übereinstimmen, ist die Authentifizierung erfolgreich.

Das Passwort wurde dabei zu keinem Zeitpunkt über das Netzwerk übertragen.

Aufgabe 3: Challenge-Response-Authentifizierungssystem

Beschreibung

Das Programm `ChallengeResponse.java` implementiert ein Challenge-Response-Authentifizierungssystem mit zwei Benutzern (`alice` und `bob`). Es verwendet SHA-256 als Hashfunktion und 128-Bit-Zufallswerte als Challenges.

Ablauf:

1. Der Client sendet seinen Benutzernamen an den Server.
2. Der Server prüft, ob der Benutzer existiert, und generiert eine zufällige Challenge.
3. Der Client berechnet die Response als `SHA-256(Challenge + Passwort)` und sendet sie zurück.
4. Der Server berechnet die erwartete Response mit dem gespeicherten Passwort und vergleicht.

Das Programm demonstriert vier Szenarien:

- Erfolgreicher Login von Alice
- Erfolgreicher Login von Bob
- Fehlgeschlagener Login (falsches Passwort)
- Fehlgeschlagener Login (unbekannter Benutzer)

Quellcode

```
1 import java.security.MessageDigest;
2 import java.security.NoSuchAlgorithmException;
3 import java.security.SecureRandom;
4 import java.util.HashMap;
5 import java.util.Map;
6
7 public class ChallengeResponse {
8
9     private final Map<String, String> users = new HashMap<>();
```

```

10 private final SecureRandom random = new SecureRandom();
11
12 public ChallengeResponse() {
13     users.put("alice", "sicheresPasswort1");
14     users.put("bob", "meinGeheimnis42");
15 }
16
17 public static void main(String[] args) throws NoSuchAlgorithmException {
18     ChallengeResponse system = new ChallengeResponse();
19
20     System.out.println("=== Challenge-Response Authentifizierung ===\n");
21
22     system.authenticate("alice", "sicheresPasswort1");
23     system.authenticate("bob", "meinGeheimnis42");
24     system.authenticate("alice", "falschesPasswort");
25     system.authenticate("eve", "irgendwas");
26 }
27
28 public void authenticate(String username, String password) throws NoSuchAlgorithmException
29 {
30     System.out.println("--- Login-Versuch: " + username + " ---");
31     System.out.println("1. Client sendet Benutzername: " + username);
32
33     if (!users.containsKey(username)) {
34         System.out.println("    Server: Benutzer nicht gefunden.");
35         System.out.println("    => Authentifizierung FEHLGESCHLAGEN\n");
36         return;
37     }
38
39     String challenge = generateChallenge();
40     System.out.println("2. Server sendet Challenge: " + challenge);
41
42     String response = computeHash(challenge + password);
43     System.out.println("3. Client berechnet Response: SHA-256(Challenge + Passwort)");
44     System.out.println("    Client sendet Response: " + response);
45
46     String expected = computeHash(challenge + users.get(username));
47     System.out.println("4. Server berechnet erwartet: " + expected);
48
49     if (response.equals(expected)) {
50         System.out.println("    => Authentifizierung ERFOLGREICH\n");
51     } else {
52         System.out.println("    => Authentifizierung FEHLGESCHLAGEN\n");
53     }
54 }
55
56 private String generateChallenge() {
57     byte[] bytes = new byte[16];
58     random.nextBytes(bytes);
59     return byteArrayToHex(bytes);
60 }
61
62 private static String computeHash(String input) throws NoSuchAlgorithmException {
63     MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
64     sha256.update(input.getBytes());
65     return byteArrayToHex(sha256.digest());
66 }
67
68 private static String byteArrayToHex(byte[] a) {

```

```
68     StringBuilder sb = new StringBuilder(a.length * 2);
69     for (byte b : a)
70         sb.append(String.format("%02x", b));
71     return sb.toString().toUpperCase();
72 }
73 }
```

Code Listing 1: ChallengeResponse.java

Programmausgabe

```
1  === Challenge-Response Authentifizierung ===
2
3  --- Login-Versuch: alice ---
4  1. Client sendet Benutzername: alice
5  2. Server sendet Challenge: A63B5C32792ED9BEDFB658C11AF2C855
6  3. Client berechnet Response: SHA-256(Challenge + Passwort)
7     Client sendet Response: 4C31CE638922C834...079039EE
8  4. Server berechnet erwartet: 4C31CE638922C834...079039EE
9     => Authentifizierung ERFOLGREICH
10
11 --- Login-Versuch: bob ---
12 1. Client sendet Benutzername: bob
13 2. Server sendet Challenge: 9B1F0EA5880C85C3AF9F6809B0DBC3FE
14 3. Client berechnet Response: SHA-256(Challenge + Passwort)
15     Client sendet Response: E4EDAFDFA2F76760...4E7A410A
16 4. Server berechnet erwartet: E4EDAFDFA2F76760...4E7A410A
17     => Authentifizierung ERFOLGREICH
18
19 --- Login-Versuch: alice (falsches Passwort) ---
20 1. Client sendet Benutzername: alice
21 2. Server sendet Challenge: EEB72B2478BAE32F3496493D5C208823
22 3. Client berechnet Response: SHA-256(Challenge + Passwort)
23     Client sendet Response: 01CB40BE53DD4766...359C6F73
24 4. Server berechnet erwartet: FEBC072BD798A8AF...AFDFC9F6
25     => Authentifizierung FEHLGESCHLAGEN
26
27 --- Login-Versuch: eve (unbekannt) ---
28 1. Client sendet Benutzername: eve
29     Server: Benutzer nicht gefunden.
30     => Authentifizierung FEHLGESCHLAGEN
```

Code Listing 2: Ausgabe des Programms